When Does Seasonal Decomposition Help? A Feature-Driven Meta-Learning Study for Time Series Forecasting

Diogo Costa[1*]

[1*]Faculty of Engineering, University of Porto, Rua Dr. Roberto Frias, Porto, 4200-465, Portugal.

Corresponding author(s). E-mail(s): diogocosta1104@gmail.com;

Abstract

Seasonal decomposition is one of the oldest preprocessing tools in time series forecasting, yet practitioners receive little guidance on when it actually helps. We study this question empirically on 3,000 monthly and quarterly series from the M3 and M4 competitions. Each series is described by six interpretable features that capture non-normality, nonlinearity, spectral entropy, evolving seasonality, structural breaks, and conditional heteroscedasticity. We compare three strategies across ten automatically tuned models from statistical, machine learning, neural, and transformer families: forecasting the raw series, forecasting the seasonally adjusted series with a naive seasonal add-back, and forecasting every STL component with a separate model. We also test whether global models trained on feature-homogeneous subsets of series outperform a single global model trained on everything. The effect of decomposition turns out to be strongly family specific. Transformers benefit consistently, with win rates near 68 percent. Statistical models are harmed because they already model trend and seasonality internally. Machine learning and neural models improve on most series but fail badly on a minority, which pulls their average performance below the naive baseline. Feature-based specialisation improves non-linear models and harms linear ones, and pairing it with seasonal adjustment removes most catastrophic failures. The results translate into simple, evidence-based rules for deciding when to decompose.

Keywords: Time series forecasting, Seasonal decomposition, STL, Meta-learning, Feature engineering, AutoML, Global forecasting models

1 Introduction

Seasonal structure dominates many of the series that organisations forecast every day, from retail sales to energy demand. A long-standing recipe handles it in two steps: decompose the series, forecast the parts that are easier to model, and add the seasonality back at the end. Seasonal and trend decomposition using Loess, known as STL [1], has been the workhorse of this recipe for three decades. It is robust, fast, implemented in every major forecasting library, and easy to explain.

Whether the recipe still pays off is much less clear. Classical statistical methods such as exponential smoothing and ARIMA model trend and seasonality internally, so an external decomposition may simply duplicate work. Global machine learning and deep learning models train on many series at once and have no built-in notion of seasonality, so they might benefit from the cleaner targets that decomposition produces. The winning method of the M4 competition relied on internal deseasonalisation [2], and recent neural architectures embed decomposition blocks directly into the network [3, 4]. Practice has clearly absorbed the idea that decomposition matters. Evidence about when it helps, for which models, and on which series remains thin.

This paper studies that question empirically. We describe each series by six interpretable features computed before any forecasting model is fitted: non-normality, nonlinearity, spectral entropy, evolving seasonality, structural break strength, and conditional heteroscedasticity. We then evaluate three decomposition strategies, from no decomposition at all to forecasting every STL component with a separate model, across ten automatically tuned models from statistical, machine learning, neural, and transformer families. The testbed contains 3,000 monthly and quarterly series sampled from the M3 and M4 competitions in a way that preserves their feature distributions. A second experiment asks a complementary question: if series are grouped by feature level, does a global model trained only on one group beat a global model trained on everything?

The study makes four contributions.

1. A systematic measurement of the effect of STL preprocessing across model families, showing that the effect is strongly family specific. Transformers gain, statistical models lose, and machine learning and neural models improve on most series while failing badly on a minority.
2. An analysis of feature-conditioned training showing that specialisation helps nonlinear models, harms linear ones, and is safest when combined with seasonal adjustment.
3. Simple evidence-based rules that practitioners can apply directly, collected in Section 4.6 and discussed in Section 5.
4. A documented failure mode of tercile-based feature bucketing when a feature has a degenerate distribution, together with the guard that prevents it.

The rest of the paper is organised as follows. Section 2 reviews related work. Section 3 describes the data, the features, the decomposition strategies, the models, and the evaluation protocol. Section 4 presents the results. Section 5 interprets them and states their limitations, and Section 6 concludes.

2 Related Work

Three lines of research meet in this study: decomposition-based forecasting, global forecasting models, and feature-based meta-learning.

Decomposition has a long history as a preprocessing step. Zhang and Qi [5] showed that neural networks struggle on raw seasonal series and improve substantially after deseasonalisation and detrending. Theodosiou [6] found decomposition-based forecasting competitive on monthly and quarterly M-competition data, the same setting we study. Bergmeir et al. [7] used STL to build bootstrap ensembles for exponential smoothing, exploiting the decomposition for variance reduction rather than direct forecasting. The idea has since moved inside model architectures. Autoformer [3] interleaves decomposition blocks with attention, and Zeng et al. [4] pair a simple trend and remainder split with linear layers and outperform far larger transformers on long-horizon benchmarks. Our study differs from all of these in treating the decomposition decision itself as the object of analysis rather than fixing it in advance.

The second line concerns global models, which train a single model across many series. Smyl [2] won the M4 competition with a hybrid that learns globally while deseasonalising each series individually. Hewamalage et al. [8] review recurrent global models and note that their behaviour depends strongly on how homogeneous the training pool is. Bandara et al. [9] made that observation operational by training recurrent networks on clusters of similar series and reported consistent gains. Our bucket-specialised training follows the same intuition, but the groups are defined by interpretable features rather than learned clusters, which makes them reproducible and easy to describe.

The third line is meta-learning for forecasting. Wang et al. [10] induced rules that link series characteristics to the best forecasting method. Kang et al. [11] placed the M3 series in a feature-based instance space to expose where methods succeed and fail. Talagala et al. [12] trained a classifier that selects a method from series features, and FFORMA [13] extended the idea to feature-based weighting of a model pool, finishing second in M4. All of this work selects or combines models. The preprocessing decision, decompose or not, is made silently before model selection even starts, and to our knowledge it has not been studied as a feature-conditioned decision in its own right.

We deliberately restrict the study to STL. It is the most widely deployed decomposition method, its robust variant handles outliers, and its additive components recompose by simple addition, which keeps the experimental design clean. The conclusions therefore concern STL specifically. Section 5 discusses what would be needed to extend them to other decomposition families.

3 Methodology

Our pipeline has five stages: sample series from the benchmark pools, compute features, decompose, forecast under every combination of decomposition strategy and model, and evaluate everything under a common protocol. This section describes each stage in turn.

3.1 Problem Formulation

Let $\mathbf{y} = (y_1, y_2, \ldots, y_T)$ be a univariate time series with seasonal period m and forecast horizon h. Given a set of forecasting methods $\mathcal{M}$ and a set of decomposition strategies $\mathcal{D}$, we ask whether observable properties of $\mathbf{y}$ can predict which pair $(d, M) \in \mathcal{D} \times \mathcal{M}$ minimises forecast error for that series.

To study this, we describe each series by a six-dimensional feature vector $\phi(\mathbf{y})$, partition the series into tercile groups along each feature dimension, train every combination of strategy and model within each group, and compare performance across groups. If decomposition helps only under certain conditions, the signal should appear as a systematic performance difference between feature terciles.

3.2 Datasets

We use two established benchmarks from the M-Competition series, the M3 [14] and M4 [15] datasets, restricted to monthly and quarterly frequencies. Table 1 summarises the collection.

Table 1 Dataset summary. Horizon and season length follow the competition definitions. Series counts reflect the representative sample described in Section 3.2.1.

Dataset	**Frequency**	**Season** m	**Horizon** h	**Series**
M3 Monthly	Monthly	12	18	750
M4 Monthly	Monthly	12	18	750
M3 Quarterly	Quarterly	4	8	750
M4 Quarterly	Quarterly	4	8	750
Total				3,000

3.2.1 Representative Sampling.

The M3 and M4 pools contain several thousand series per frequency, so we draw 750 series per source dataset with stratified quantile sampling. Series are ranked by each feature ϕ_k and divided into $S = 30$ strata of equal size. The quota of 750 series is distributed across strata proportionally and filled by draws without replacement, so the sample reproduces the marginal feature distribution of the full pool. A fixed random seed makes the sampling deterministic.

3.3 Time Series Feature Extraction

Six features characterise each series. Where noted, they operate on the STL residuals described in Section 3.4, so that they capture intrinsic behaviour rather than removable seasonal structure. All features are computed from training observations only, with no access to the test period. Four of the six features are computed from STL components,

which ties the features to the decomposition under study. Section 5 returns to this point.

Non-normality, ϕ_1**.**

Residuals r_t from the STL fit are standardised as $z_t = (r_t - \tilde{r})/(1.4826 \cdot \mathrm{MAD}(r) + \varepsilon)$, where $\tilde{r}$ is the median. We apply the D'Agostino omnibus normality test [16] to $\{z_t\}$ and convert its p value to a log-scale score:

$$\phi_1 = \min\bigl(-\log_{10}(p_1 + 10^{-12}),\ 12\bigr). \tag{1}$$

High values indicate strong departure from Gaussian residuals.

Nonlinearity, ϕ_2**.**

We test whether the lagged history contains nonlinear predictive structure by comparing a linear and a nonlinear autoregressive model. Let $\ell = \min(m, 12)$ be the embedding dimension. We build lag-feature matrices from $\{(y_{t-\ell}, \ldots, y_{t-1}, y_t)\}$ and split the data chronologically at 55, 70, and 85 percent of the series length to obtain three train and test folds. On each fold we fit a ridge regression [17] with $\alpha = 1$ and a random forest [18] with 40 trees of maximum depth four. The feature is the mean relative improvement of the nonlinear model:

$$\phi_2 = \min\left(\max\left(0,\ \frac{\overline{\mathrm{MAE}}_{\mathrm{linear}} - \overline{\mathrm{MAE}}_{\mathrm{RF}}}{\overline{\mathrm{MAE}}_{\mathrm{linear}} + 10^{-8}}\right),\ 2\right), \tag{2}$$

where $\overline{\mathrm{MAE}}$ is the mean across folds. In plain terms, ϕ_2 is the relative error reduction of the random forest over ridge regression, floored at zero and capped at two. A value of zero means the random forest offers no improvement, which is structurally common in linear datasets, as Section 4.4 shows.

Spectral entropy, ϕ_3**.**

We compute the power spectral density of the demeaned series via a periodogram and derive its normalised Shannon entropy:

$$\phi_3 = -\frac{\sum_k p_k \ln p_k}{\ln K}, \tag{3}$$

where $p_k = \hat{S}(f_k)/\sum_j \hat{S}(f_j)$ are the normalised spectral weights and K is the number of non-zero frequencies. The result lies in $[0, 1]$. Low values indicate a spectrum concentrated at a few dominant frequencies, which means strong and regular periodicity. High values indicate a diffuse, unpredictable spectrum [11].

Evolving seasonality, ϕ_4**.**

To detect seasonal behaviour that changes over time, we compute the seasonal strength of Equation 4 in rolling windows of length $\max(36, 3m)$ for monthly and $\max(16, 4m)$

for quarterly series, advancing by $\lfloor m/2 \rfloor$ observations per step. The seasonal strength in each window is

$$F_s^{(w)} = \max\left(0,\ 1 - \frac{\mathrm{Var}(R^{(w)})}{\mathrm{Var}(S^{(w)} + R^{(w)})}\right), \tag{4}$$

where $S^{(w)}$ and $R^{(w)}$ are the STL seasonal and residual components in window w. The feature is the standard deviation of the sequence $\{F_s^{(w)}\}$. A high value indicates that seasonal strength changes substantially over time.

Structural break strength, ϕ_5***.***

We detect level shifts in the nonseasonal component $T_t + R_t$ with the PELT algorithm [19], using an RBF cost function and a penalty proportional to $\log(n)$. Let $\mathcal{B}$ be the set of breakpoints detected in the central 60 percent of the series, which avoids boundary effects. For each candidate break $b \in \mathcal{B}$, the mean-shift contrast is

$$\Delta_b = \frac{|\bar{x}_{<b} - \bar{x}_{\geq b}|}{1.4826 \cdot \mathrm{MAD}(T + R) + \varepsilon}, \tag{5}$$

and $\phi_5 = \min(\max_b \Delta_b,\ 20)$. The feature measures the size of the largest structural shift relative to the scale of the nonseasonal component.

Conditional heteroscedasticity, ϕ_6***.***

We test for ARCH effects [20] in the normalised STL residuals $\{z_t\}$ using Engle's LM test with lag order $\min(12, \lfloor n/5 \rfloor)$ and return

$$\phi_6 = \min\big(-\log_{10}(p_6 + 10^{-12}),\ 12\big), \tag{6}$$

where p_6 is the test p value. High values indicate serially correlated residual variance, a pattern known as volatility clustering.

3.3.1 Feature Bucket Assignment.

For each feature ϕ_k and frequency, all selected series are ranked by feature value and assigned to one of three buckets of equal size. Low contains the bottom third, Medium the middle third, and High the top third. The bucket boundaries are the 33.3^{rd} and 66.7^{th} percentiles computed across the combined M3 and M4 series. This partitioning drives the bucket-specialised training described in Section 3.6.

3.4 STL Decomposition Framework

Seasonal and trend decomposition using Loess, hereafter STL [1], factorises a time series as

$$y_t = T_t + S_t + R_t, \tag{7}$$

where T_t is a locally smooth trend, S_t is a periodic seasonal component that is approximately zero-mean within each period, and R_t is the remainder. We use the robust

variant of STL, which down-weights outlier observations during the Loess smoothing passes, as implemented in `statsmodels` [21].

We evaluate three strategies for using this decomposition.

Direct. The forecasting model operates on the raw series $\mathbf{y}$ and no decomposition is applied.

STL Seasonal Naive. The model is trained on the nonseasonal component $N_t = T_t + R_t$ and produces a forecast $\hat{N}_{t+1:t+h}$. The seasonal component is extrapolated by repeating the last observed full seasonal cycle, $\hat{S}_{t+j} = S_{t+j-m}$ for $j = 1, \ldots, h$. The final forecast is $\hat{y}_{t+j} = \hat{N}_{t+j} + \hat{S}_{t+j}$.

STL All Components. Three separate model instances are trained on T_t, S_t, and R_t, yielding component forecasts $\hat{T}$, $\hat{S}$, and $\hat{R}$. The final forecast is the sum:

$$\hat{y}_{t+j} = \hat{T}_{t+j} + \hat{S}_{t+j} + \hat{R}_{t+j}, \quad j = 1, \ldots, h. \tag{8}$$

Tables abbreviate the three strategies as Direct, STL-SN, and STL-AC.

3.5 Forecasting Model Families

We benchmark ten models from four families across two training paradigms. Statistical models fit one series at a time. All other families train global models on the full cohort of series.

Statistical.

AutoARIMA [22], AutoSARIMA with seasonal differencing forced to $D = 1$, and AutoETS [23], all implemented via `statsforecast` [24].

Machine learning.

AutoRidge, AutoLinearRegression, AutoXGBoost [25], and AutoRandomForest, trained jointly on all cohort series with sliding-window lag features via `MLForecast` [26]. Hyperparameters are tuned with Optuna [27] over 100 trials.

Neural.

AutoNLinear, AutoNHITS [28], and AutoLSTM, implemented via `NeuralForecast` [29].

Transformers.

AutoTFT [30] and AutoPatchTST [31], also implemented via `NeuralForecast`.

3.6 Bucket-Specialised Training

For the global model families we investigate whether regime-specialised models outperform a single global model trained on all 750 series of a cohort. The hypothesis is that series sharing the same feature tercile have sufficiently similar dynamics to reward a dedicated model.

Each task configuration combines one feature, one frequency, one decomposition strategy, and one model. For each configuration we train three independent global models, one per tercile bucket. Each model trains exclusively on the roughly 250 series in its bucket and is evaluated exclusively on test series from the same bucket. The single global model, trained on all 750 series with no bucket restriction, serves as the comparison baseline throughout. Performance is measured with the specialisation delta Δ_{FT} defined in Section 3.7, where a positive value means the specialised model wins.

Statistical models are excluded because they fit each series independently, so the composition of the training pool cannot affect them. Every global configuration is therefore evaluated twice, once with all 750 series as the global baseline and once per tercile bucket under the specialised condition.

3.7 Evaluation Protocol

All experiments share the evaluation protocol described in this section.

3.7.1 Cross-Validation.

We use rolling-origin cross-validation [32] with $W = 3$ windows per series, advancing by h observations per window. For each series i and window w, the training set ends at a cutoff $c_i^{(w)}$ and the test set contains exactly the next h observations.

3.7.2 Metric.

The primary metric is the relative naive error, RelNaive for short:

$$\mathrm{RelNaive} = \frac{\mathrm{MAE}(\mathbf{y},\, \hat{\mathbf{y}})}{\mathrm{MAE}(\mathbf{y},\, \hat{\mathbf{y}}^{\mathrm{naive}})}, \tag{9}$$

where $\hat{\mathbf{y}}^{\mathrm{naive}}$ is the seasonal naive forecast that repeats the last observed seasonal cycle, $\hat{y}_{t+j}^{\mathrm{naive}} = y_{t+j-m}$ for $j = 1, \ldots, h$. A value below 1 means the model beats the seasonal naive baseline. We also report a clipped variant $\mathrm{RelNaive}^{(10)}$ that caps values at 10, which limits the influence of catastrophic failures on aggregate rankings. Aggregate statistics are means over all series and windows within each reporting group, where a group fixes the feature, the decomposition method, the model family, and the training scope.

The STL delta measures the improvement from applying STL decomposition relative to the Direct strategy:

$$\Delta_{\mathrm{STL}} = \mathrm{RelNaive}^{(10)}_{\mathrm{Direct}} - \mathrm{RelNaive}^{(10)}_{\mathrm{STL}}, \tag{10}$$

so a positive value means STL reduces error. The specialisation delta measures the improvement of a bucket-specialised model over the global baseline:

$$\Delta_{\mathrm{FT}} = \mathrm{RelNaive}^{(10)}_{\mathrm{global}} - \mathrm{RelNaive}^{(10)}_{\mathrm{bucket}}, \tag{11}$$

where again a positive value favours the specialised model.

We assess the statistical significance of aggregate deltas with the Wilcoxon signed-rank test [33] applied at the series level, correcting p values for multiple comparisons with Holm's method [34].

4 Results

We report results in five parts: the global baselines, the effect of STL decomposition, the effect of bucket-specialised training, an edge case in the feature bucketing, and the statistical significance of all comparisons. Section 4.6 collects the main findings.

4.1 Global Baseline Performance

All 3,000 series, 1,500 monthly and 1,500 quarterly, are evaluated over three rolling cross-validation windows. Table 2 ranks the global model configurations by mean RelNaive$^{(10)}$ over all series with no bucket restriction. AutoETS without decomposition is the best single configuration, with a mean of 0.888 and a median of 0.791, and the only configuration without preprocessing whose mean beats the naive forecast. STL-SN paired with statistical or transformer models dominates the next nine places.

Table 2 Top 10 global model configurations by mean RelNaive$^{(10)}$, where lower is better. Direct means no decomposition, STL-SN is STL Seasonal Naive, and STL-AC is STL All Components.

Rank	Family	Model	Decomp.	Mean	Median
1	Statistical	AutoETS	Direct	0.888	0.791
2	Statistical	AutoARIMA	STL-SN	0.930	0.887
3	Neural	AutoNLinear	STL-SN	0.931	0.889
4	Statistical	AutoSARIMA	STL-SN	0.931	0.889
5	ML	AutoLinearRegression	STL-SN	0.933	0.893
6	ML	AutoRidge	STL-SN	0.933	0.889
7	Transformer	AutoPatchTST	STL-SN	0.933	0.891
8	Statistical	AutoETS	STL-SN	0.935	0.918
9	Transformer	AutoPatchTST	STL-AC	0.936	0.826
10	Transformer	AutoTFT	STL-SN	0.936	0.919

A persistent divergence between mean and median runs through the whole study. Under STL decomposition the ML and Neural families have mean RelNaive$^{(10)}$ between 1.12 and 1.19 while their medians stay near or below 1. A heavy right tail of catastrophic failures on a minority of series pulls the mean above the naive baseline even when most series benefit. We therefore report both statistics throughout.

4.2 Effect of STL Decomposition

Table 3 reports the STL delta for the global model. Positive values mean STL reduces error relative to Direct.

Table 3 STL delta Δ_{STL} by decomposition and model family for the global model. Mean and median are taken over all series and windows. The win rate is the fraction of series where STL helps.

Decomposition	Family	Mean Δ	Median Δ	Win Rate
STL-AC	Transformer	+0.039	+0.141	68.5%
STL-SN	Transformer	+0.063	+0.091	67.9%
STL-AC	Neural	−0.119	+0.063	57.6%
STL-SN	Neural	−0.109	+0.027	55.4%
STL-AC	ML	−0.191	+0.023	52.5%
STL-SN	ML	−0.194	−0.002	49.5%
STL-AC	Statistical	−0.028	−0.085	39.8%
STL-SN	Statistical	−0.009	−0.109	37.1%

Transformers.

Transformers benefit robustly from STL. Both STL-SN and STL-AC give positive mean and median deltas, with win rates near 68%. The improvement is strongest under STL-AC, with a median of +0.141, which suggests that training separate global submodels for trend, seasonal, and residual components fits the inductive biases of PatchTST and TFT well.

Neural and ML models.

These families show the divergence between mean and median at its most extreme. For Neural the median is positive, +0.063 under STL-AC and +0.027 under STL-SN, but the mean is deeply negative at −0.119 and −0.109. STL helps most series but fails badly on series with non-stationary seasonal patterns, where the decomposition itself is unreliable. ML fares worse, with an STL-SN mean of −0.194 and a median of essentially zero.

Statistical models.

Both decomposition strategies harm statistical models, with win rates between 37% and 40% and negative means and medians. AutoARIMA, AutoSARIMA, and AutoETS already model trend and seasonality internally. STL preprocessing adds a redundant transformation and extra parameter uncertainty, so accuracy degrades rather than improves.

Frequency effects are marginal. The median STL-AC gain is +0.033 for monthly series and +0.024 for quarterly series, with no directional difference between the two frequencies.

4.3 Bucket-Specialised Training

This section measures whether a global model trained only on the series of one feature bucket beats the single global model trained on all series.

4.3.1 Overall Results

Across 1,449,738 series-window evaluations, the specialised models achieve a median Δ_{FT} of +0.050, a mean of −0.029, and a win rate of 56.1%. The gap between mean and median mirrors the STL results. Specialisation benefits the majority of series but fails badly on a tail, and that tail drags the mean negative.

Medium-bucket series improve most consistently, with a median of +0.059 and a win rate of 56.8%. Low and High are comparable, with medians of +0.048 and +0.046. The Medium advantage is intuitive. These are the modal series of each cohort, so the specialised model is not pulled toward the extremes present in either tail.

4.3.2 Non-Linear Versus Linear Models

The clearest split in the study separates non-linear from linear models. Table 4 aggregates over everything else.

Table 4 Specialisation delta by model type. Results aggregate over all families, buckets, decompositions, and series.

Model type	Mean Δ_{FT}	Median Δ_{FT}	Win rate
Non-linear	+0.082	+0.092	60.1%
Linear	−0.251	−0.014	48.1%

Non-linear models benefit. Focusing training on roughly 250 regime-homogeneous series improves generalisation within that regime, and both mean and median are positive, so the gain is robust. Linear models are harmed. Roughly 250 training series are too few for stable coefficient estimation, and the distribution shift induced by bucketing makes the problem worse. The win rate of 48.1% sits below chance, which signals a directional harm rather than noise. Section 4.5 confirms the significance.

Table 5 gives the per-model breakdown.

AutoXGBoost and AutoLSTM gain the most. AutoXGBoost reaches a median of +0.187 with a win rate of 65.3%, and AutoLSTM a median of +0.162 with a win rate of 64.2%. Both benefit from the focused within-bucket training distribution. AutoNHITS and the transformers have positive medians but negative means, the signature of fat-tailed error distributions. AutoRidge and AutoLinearRegression suffer the largest harm, with a mean delta of −0.327, a direct consequence of coefficient instability on small training pools. The practical rule is simple: restrict bucket specialisation to non-linear models.

4.3.3 Interaction with STL Decomposition

Specialisation also interacts with the decomposition strategy, as Table 6 shows.

Direct gives the highest median, +0.108, but the worst mean, −0.156. Without seasonal preprocessing, a model trained on roughly 250 series overfits the idiosyncratic

Table 5 Specialisation delta per model, aggregated over all buckets, decompositions, features, and frequencies. Linear models are marked L and non-linear models NL.

Family	Model	Type	Mean Δ	Median Δ	Win rate
ML	AutoXGBoost	NL	+0.236	+0.187	65.3%
Neural	AutoLSTM	NL	+0.231	+0.162	64.2%
ML	AutoRandomForest	NL	+0.146	+0.155	61.6%
Neural	AutoNHITS	NL	−0.031	+0.076	59.2%
Transformer	AutoTFT	NL	−0.076	+0.059	57.5%
Transformer	AutoPatchTST	NL	−0.013	+0.014	52.6%
ML	AutoRidge	L	−0.327	−0.004	49.5%
ML	AutoLinearRegression	L	−0.327	−0.002	49.7%
Neural	AutoNLinear	L	−0.099	−0.029	45.1%

Table 6 Specialisation delta by decomposition strategy. All non-statistical model families and buckets are combined.

Decomposition	Mean Δ_{FT}	Median Δ_{FT}	Win rate
Direct	−0.156	+0.108	60.4%
STL-SN	+0.044	+0.032	55.6%
STL-AC	+0.026	+0.021	52.2%

structure of its bucket. STL-SN is the most robust pairing, with a mean of +0.044 and a median of +0.032. Removing seasonality first gives the model a more stationary target and prevents most catastrophic failures.

4.4 The Nonlinearity Edge Case

The nonlinearity feature ϕ_2 exposes a structural limitation of tercile bucketing. Because ϕ_2 is floored at zero in Equation 2, the M3 and M4 series accumulate at exactly zero: 81.8% of monthly and 92.8% of quarterly series. Both tercile boundaries therefore coincide at $q_{1/3} = q_{2/3} = 0$. Every zero-valued series falls into Low, every non-zero series falls into High, and the Medium bucket is empty. Table 7 shows the resulting bucket sizes.

Table 7 Bucket sizes for the nonlinearity feature ϕ_2. The coincident boundaries $q_{1/3} = q_{2/3} = 0$ force a binary split into Low and High.

Frequency	Low	Medium	High	Total
Monthly	1,227 (81.8%)	**0**	273 (18.2%)	1,500
Quarterly	1,392 (92.8%)	**0**	108 (7.2%)	1,500

The 27 affected task configurations, 1.9% of all bucket tasks, are excluded from comparative analyses. The binary split that remains is still meaningful. Low collects the series where lagged ridge regression is near optimal, and High collects the minority with genuinely exploitable nonlinear structure. This is in fact the sharpest regime contrast among all six features.

4.5 Statistical Significance

We applied Wilcoxon signed-rank tests with Holm correction to every combination of model, decomposition, and bucket, which gives $n = 81$ tests. Of these, 55 combinations improve significantly and 23 degrade significantly, all with $p_{\text{Holm}} < 0.001$, while 3 show no significant change. In relative terms, specialisation helps significantly in 67.9% of configurations and harms significantly in 28.4%.

The five most improved combinations all pair a non-linear model with the Medium bucket and STL decomposition, as Table 8 shows. The combination of STL-SN, the Medium bucket, and AutoXGBoost is the single strongest result in the study, with a median Δ_{FT} of +0.334 and a win rate of 72.3%.

Table 8 Top 5 significantly improved combinations of model and bucket. All pass the Wilcoxon test with Holm correction at $p_{\text{Holm}} < 0.001$.

Family	Model	Decomp.	Bucket	Med. Δ	Win rate
ML	AutoXGBoost	STL-SN	Medium	+0.334	72.3%
ML	AutoXGBoost	STL-AC	Medium	+0.308	66.9%
Neural	AutoLSTM	STL-SN	Medium	+0.299	69.2%
ML	AutoRandomForest	STL-AC	Medium	+0.291	64.4%
ML	AutoRandomForest	STL-SN	Medium	+0.290	68.2%

The significantly harmed combinations are exclusively the linear models and AutoNLinear. The worst case is AutoNLinear under STL-AC, with a median Δ_{FT} of -0.088 and a win rate of 34.5%. Linear models under STL-AC suffer twice over. The global model must fit trend, seasonal, and residual components of roughly 250 series at once, which leaves each linear submodel too few degrees of freedom while recomposition compounds the errors.

4.6 Summary of Findings

Four findings summarise the study.

1. STL decomposition effects are family specific, as Section 4.2 shows. Transformers benefit robustly, with positive mean and median deltas and win rates near 68%. Neural and ML models show positive medians but severely negative means, because STL fails on non-stationary series. Statistical models are uniformly harmed, since their internal trend and seasonal modelling makes STL preprocessing redundant.
2. Bucket specialisation improves non-linear models and harms linear ones, as Section 4.3.2 shows. Non-linear architectures gain a median of +0.092 with a win

rate of 60.1%, while linear models lose a median of −0.014 with a win rate of 48.1%. The split is statistically significant in 67.9% of tested configurations.

3. The Medium bucket is the most reliable and STL-SN is the safest pairing, as Sections 4.3.1 and 4.3.3 show. Medium-bucket series yield the most consistent specialisation gains, and applying STL-SN before bucket training gives positive mean and median deltas, eliminating the catastrophic failures seen under Direct.
4. Tercile bucketing collapses to a binary split when a feature has a degenerate distribution, as Section 4.4 shows. The zero floor of ϕ_2 empties the Medium bucket for 1.9% of tasks. Tercile partitioning needs a minimum-support guard.

5 Discussion

The central message of the study is that the value of seasonal decomposition depends far more on the forecasting model than is usually acknowledged. Statistical methods lose accuracy under STL preprocessing because they already estimate trend and seasonal structure internally, so the external decomposition adds a redundant transformation and a second source of estimation error. Transformers gain because the decomposed components match their inductive biases, and training one global submodel per component appears to act as a useful division of labour. Machine learning and neural models sit in between. They benefit on the majority of series, but they depend on the decomposition being right, and when seasonality drifts over time the seasonal naive extrapolation breaks and the recomposed forecast can fail badly.

The same asymmetry explains the persistent gap between means and medians. Decomposition and specialisation both improve typical performance while occasionally producing catastrophic failures, and a handful of failures is enough to reverse a ranking based on means. Any study or production system that evaluates preprocessing decisions on averages alone risks drawing the wrong conclusion. We recommend reporting medians and win rates next to means as a matter of course.

For practitioners the rules that emerge are short. Do not decompose for statistical models. Decompose for transformers, ideally modelling all components separately. If you train regime-specialised global models, restrict them to non-linear architectures, remove seasonality first with the seasonal naive add-back, and expect the largest gains on series in the middle of the feature distribution. Treat linear global models trained on small pools with suspicion.

Several limitations bound these conclusions. The study covers one decomposition method. STL is the most widely used choice and its additive components recompose trivially, but other families such as wavelet decompositions may behave differently and deserve the same treatment. Four of the six features are computed from STL components, so the features and the treatment share machinery. A series that STL fits poorly receives both an unreliable decomposition and an unusual feature value, which makes part of the observed association self-referential. The study also stops one step short of full meta-learning, since we never train a selector that maps features to a recommended strategy. Finally, the evidence comes from monthly and quarterly M3 and M4 data with pools of roughly 250 series per bucket, and other frequencies, domains, or pool sizes may shift the boundaries we report.

These limitations point directly at future work. The most natural extensions are repeating the design with other decomposition methods, training an explicit meta-learner on the six features, and testing whether pretrained foundation models for time series respond to deseasonalised inputs the way transformers trained from scratch do.

6 Conclusion

We asked when seasonal decomposition helps time series forecasting and answered with a controlled study of 3,000 monthly and quarterly M3 and M4 series, six interpretable features, three decomposition strategies, and ten automatically tuned models. The answer depends mostly on the model family. Transformers benefit consistently from STL preprocessing, statistical models are consistently harmed, and machine learning and neural models gain on most series while failing badly on a minority with unstable seasonality. Training separate global models per feature bucket helps non-linear architectures and harms linear ones, and pairing specialisation with seasonal adjustment removes most of its catastrophic failures.

Beyond the specific numbers, the study makes a broader point. The decision to decompose is a modelling decision like any other, and it deserves the same empirical scrutiny that model selection already receives. Series features computed before any model is fitted offer a cheap and interpretable way to apply that scrutiny, and our results show they carry real signal about when preprocessing will pay off.

Acknowledgements. Not applicable.

Declarations

- **Funding:** Not applicable.
- **Conflict of interest:** The author declares no competing interests.
- **Ethics approval:** Not applicable.
- **Data availability:** The M3 and M4 datasets are publicly available at https://forecasters.org/resources/time-series-data/.
- **Code availability:** Source code will be made available upon acceptance.
- **Author contribution:** D.C. conceived and designed the study, implemented the pipeline, ran all experiments, and wrote the manuscript.

References

[1] Cleveland, R.B., Cleveland, W.S., McRae, J.E., Terpenning, I.: STL: A seasonal-trend decomposition procedure based on loess. Journal of Official Statistics **6**(1), 3–73 (1990)

[2] Smyl, S.: A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting. International Journal of Forecasting **36**(1), 75–85 (2020)

[3] Wu, H., Xu, J., Wang, J., Long, M.: Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In: Advances in Neural Information Processing Systems, vol. 34, pp. 22419–22430 (2021)

[4] Zeng, A., Chen, M., Zhang, L., Xu, Q.: Are transformers effective for time series forecasting? In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, pp. 11121–11128 (2023)

[5] Zhang, G.P., Qi, M.: Neural network forecasting for seasonal and trend time series. European Journal of Operational Research **160**(2), 501–514 (2005)

[6] Theodosiou, M.: Forecasting monthly and quarterly time series using STL decomposition. International Journal of Forecasting **27**(4), 1178–1195 (2011)

[7] Bergmeir, C., Hyndman, R.J., Benítez, J.M.: Bagging exponential smoothing methods using STL decomposition and Box–Cox transformation. International Journal of Forecasting **32**(2), 303–312 (2016)

[8] Hewamalage, H., Bergmeir, C., Bandara, K.: Recurrent neural networks for time series forecasting: Current status and future directions. International Journal of Forecasting **37**(1), 388–427 (2021)

[9] Bandara, K., Bergmeir, C., Smyl, S.: Forecasting across time series databases using recurrent neural networks on groups of similar series: A clustering approach. Expert Systems with Applications **140**, 112896 (2020)

[10] Wang, X., Smith-Miles, K., Hyndman, R.J.: Rule induction for forecasting method selection: Meta-learning the characteristics of univariate time series. Neurocomputing **72**(10–12), 2581–2594 (2009)

[11] Kang, Y., Hyndman, R.J., Smith-Miles, K.: Visualising forecasting algorithm performance using time series instance spaces. International Journal of Forecasting **33**(2), 345–358 (2017)

[12] Talagala, T.S., Hyndman, R.J., Athanasopoulos, G.: Meta-learning how to forecast time series. Journal of Forecasting **42**(6), 1476–1501 (2023)

[13] Montero-Manso, P., Athanasopoulos, G., Hyndman, R.J., Talagala, T.S.: FFORMA: Feature-based forecast model averaging. International Journal of Forecasting **36**(1), 86–92 (2020)

[14] Makridakis, S., Hibon, M.: The M3-Competition: Results, conclusions and implications. International Journal of Forecasting **16**(4), 451–476 (2000)

[15] Makridakis, S., Spiliotis, E., Assimakopoulos, V.: The M4 competition: 100,000 time series and 61 forecasting methods. International Journal of Forecasting **36**(1), 54–74 (2020)

[16] D'Agostino, R.B.: An omnibus test of normality for moderate and large size samples. Biometrika **58**(2), 341–348 (1971)

[17] Hoerl, A.E., Kennard, R.W.: Ridge regression: Biased estimation for nonorthogonal problems. Technometrics **12**(1), 55–67 (1970)

[18] Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001)

[19] Killick, R., Fearnhead, P., Eckley, I.A.: Optimal detection of changepoints with a linear computational cost. Journal of the American Statistical Association **107**(500), 1590–1598 (2012)

[20] Engle, R.F.: Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. Econometrica **50**(4), 987–1007 (1982)

[21] Seabold, S., Perktold, J.: Statsmodels: Econometric and statistical modeling with Python. In: 9th Python in Science Conference, pp. 92–96 (2010)

[22] Hyndman, R.J., Khandakar, Y.: Automatic time series forecasting: The `forecast` package for R. Journal of Statistical Software **27**(3), 1–22 (2008)

[23] Hyndman, R.J., Koehler, A.B., Ord, J.K., Snyder, R.D.: Forecasting with Exponential Smoothing: The State Space Approach. Springer, ??? (2008)

[24] Olivares, K.G., Challu, C., Garza, F., Mergenthaler, M., Nixtla: statsforecast: Lightning fast forecasting with statistical and econometric models. arXiv preprint arXiv:2207.03175 (2022)

[25] Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794 (2016)

[26] Garza, F., Canseco, M.M., Challú, C., Olivares, K.G.: Mlforecast: Scalable machine learning for time series forecasting. arXiv preprint arXiv:2308.09372 (2022)

[27] Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 2623–2631 (2019)

[28] Challu, C., Olivares, K.G., Oreshkin, B.N., Garza, F., Mergenthaler-Canseco, M., Dubrawski, A.: N-HiTS: Neural hierarchical interpolation for time series forecasting. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, pp. 6989–6997 (2023)

[29] Olivares, K.G., Challu, C., Garza, F., Canseco, M.M., Dubrawski, A.: Neuralforecast: User friendly state-of-the-art neural forecasting models. arXiv preprint

arXiv:2304.02673 (2023)

[30] Lim, B., Arık, S.Ö., Loeff, N., Pfister, T.: Temporal fusion transformers for interpretable multi-horizon time series forecasting. International Journal of Forecasting **37**(4), 1748–1764 (2021)

[31] Nie, Y., H. Nguyen, N., Sinthong, P., Kalagnanam, J.: A time series is worth 64 words: Long-term forecasting with transformers. In: International Conference on Learning Representations (2023)

[32] Tashman, L.J.: Out-of-sample tests of forecasting accuracy: An analysis and review. International Journal of Forecasting **16**(4), 437–450 (2000)

[33] Wilcoxon, F.: Individual comparisons by ranking methods. Biometrics Bulletin **1**(6), 80–83 (1945)

[34] Holm, S.: A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics **6**(2), 65–70 (1979)